THE EDITOR OF THE SUITE

bm Animator: rigs and animations

Give a 3D model a skeleton, make it walk, and use it in your game or as pre-rendered sprites.

Contents

What bm Animator is 4

What it does 4

How it connects to Studio and Mesh 4

Which editor for what 4

Good to know 4

Opening 5

Three ways in 5

Projects and games 5

Good to know 5

Try it, step by step 5

The screen and the controls 6

Pages and global keys 6

F1, play 6

F2, rig 7

F3, animate 7

F4, sprites 8

Gamepad 9

Caveats 9

Bones and rigging 10

What it does 10

Why it matters 10

Good to know 10

The skin 10

Try it, step by step 10

Keyframes and clips 12

What it does 12

Why it matters 12

Good to know 12

Onion skin 12

Try it, step by step . . . 12

Hitboxes and baking sprites (F4) . . . 14

What it does . . . 14

The settings . . . 14

The bone hitboxes . . . 14

Putting the sprites in the sheet . . . 15

Good to know . . . 15

Try it, step by step . . . 15

Saving and exporting . . . 16

What it does . . . 16

Good to know . . . 16

Try it, step by step . . . 16

Import a .glb . . . 16

Using the animation in a game . . . 18

The 3D model . . . 18

The 2D sprites . . . 18

Code to copy . . . 18

Tutorials . . . 20

Rig a model . . . 20

Make a walk cycle . . . 20

Play it in a game . . . 20

Tips and limits . . . 22

Tips . . . 22

Limits . . . 22

What bm Animator does not do . . . 22

Caveats . . . 22

Open points . . . 23

README

What this is. The manual of **bm Animator**, the editor of skeletons (rigs) and animations. Every key in it comes from the editor's own code.

How to use it. Read "Opening" and "The screen and the controls" once, then follow the three tutorials with a model of yours.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

What bm Animator is

bm Animator puts a skeleton into a 3D model, moves the bones on a timeline and plays the result. It is built into the console: you find it in the **Dev** tab.

What it does

- It plays the animations of a model and lets you check them (page F1).
- It builds the skeleton: bones, parents, mirrored sides and the skin that ties the faces to the bones (page F2).
- It records animations as keyframes on a timeline of 12 frames per second (page F3).
- It draws an animation into the sprite sheet as pre-rendered 2D sprites, with hitboxes taken from the bones (page F4).

How it connects to Studio and Mesh

- **bm Studio** makes the models. bm Animator does not build models: it opens the ones of the file. In its menu, **Open in bm Studio** saves and switches to the same file.
- **bm Mesh** edits the vertices and faces of the same models. It keeps the skeleton of a model, so bones and animations stay those of bm Animator.
- Models live in the MESH section of the cartridge, skeletons and animations in the ANIM section. bm Animator changes only those two sections (and the sprite sheet, when you put sprites in it).
- The game uses the result with `model()` and `animate()`.

Which editor for what

- **bm Studio**: static models. Objects, props and buildings, made of blocks and tiles. The 3D map of a game is made in the map page of bm SDK.
- **bm Mesh**: organic models. Characters and heroes, and all fine editing of vertices and triangles.
- **bm Animator**: rigs and animations. It puts a skeleton into a model and moves it. It does not build models.
- bm Studio and bm Mesh write the same MESH section, so a model can go from one editor to the other.

Good to know

- The Pi Zero W has bm Animator. The kernel of the RGB30 does not have it: it does not offer the suite's programs.
- You work on a **project** (`.bme`), never on a game: see "Opening".

Opening

Three ways in

- From the **Dev** tab of the menu: start **bm Animator** and choose a file.
- From a game: press **X** on its cover and choose **Open in bm Animator**.
- From another tool of the suite: the menu entry **Open in bm Animator** (bm Studio has it) switches to the same file.

If the file list is empty, the screen says `no .bm files: make models in bm Studio`. If you open a file with no 3D models, it says `no models: make them in bm Studio`. The text `open a .bm with 3D models (made in bm Studio)` is shown when the app starts with no file and there are `.bm` files.

Projects and games

File	What it is	What bm Animator does with it
`.bme`	a project	opens it, changes it, saves it in its place
`.bm` , `.b16`	a game	only reads it

With a game open, the first save asks `Save an editable copy: name.bme?`. Answer yes: from then on you work on the copy, in `/bme`. The game itself is never changed.

Good to know

- The list of files shows the `.bme` projects of `/bme` and the `.bm` and `.bme` files of `/carts` and of the root of the SD card.
- The menu page shows the file name and `*modified*` when there is something to save.

Try it, step by step

1. Start the console and open the **Dev** tab.
2. Start **bm Animator**.
3. Choose a `.bm` or `.bme` with a model in it.
4. You see page **F1** with the model turning. The lists on the left are MODELS and ANIMATIONS, with names only; the name of the model and its vertices, triangles and bones are in the info strip to the right of the panel.

TRY IT

Press `]` and `[` to move to the next and the previous model of the file. The status line says `model` and the name.

The screen and the controls

The layout is the one of the other console apps: on the left a panel with lists, above the 3D view two lines that say what is there and what is being done, at the bottom the keys of the page. Chosen things are in yellow. Hold **F12**, or press **?**, to see the keys of the page.

[TO FILL: shots/editor-animator-play.png]

Pages and global keys

Key	What it does
`F1`	play: the player
`F2`	rig: bones and skin
`F3`	animate: keyframes
`F4`	sprites: an animation drawn into the sprite sheet
`Esc`	the menu (back from it)
`Ctrl+S`	save
`Ctrl+Shift+S`	save as (8.3 name in `/carts`)
`Ctrl+O`	open another file
`Ctrl+I`	import a model from `/bmm` or `/glb`
`Ctrl+E`	export the model to `/bmm`
`F5` or `Ctrl+R`	try the game (saves first, then comes back to the same page)
`F6`	the assistant, where the kernel has it
`Ctrl+Z`, `Ctrl+Y`	undo, redo
`[` , `]`	previous, next model
`F12` held, or `?`	show the keys of the page
`+` , `-`	zoom the 3D view
`Alt` + arrows	turn the camera

F1, play

Key	What it does
up, down	previous, next model
left, right	previous, next animation
space or Enter	play, stop
`a` `d` `w` `s`	turn the camera by hand (`a` and `d` stop the automatic spin; `w` and `s` only tilt)

Key	What it does
`o`	the camera turns by itself on, off
`k`	show the skeleton
`l`	light on, off
`b`	blend the animation with the next one: 0, 25, 50, 75 %
`,` `.`	one frame back, forward (stops playing)
`<` `>`	slower, faster (from 1/8 to 4 times)
`z` or `f`	frame the model

F2, rig

Key	What it does
`n`	make the skeleton, or add a bone under the chosen one
up, down	choose the bone
`Tab`	work on the bone's tail or on its head
`j`	head alone: **w a s d r f** move only the head of the bone, off the tail of its parent (press again: joints go together)
`w` `a` `s` `d` `r` `f`	move the chosen end by 1/8 (uppercase: 1/32)
`m`	mirror the bone and its children to the other side
`p`	choose the parent
Enter	rename the bone (up to 16 letters)
`x` or Del	delete the bone
`k`	skin: each face follows its nearest bone
`K`	skin: each corner follows its nearest bone (smooth)
`v`	skin mode: choose faces by hand
`t`	test pose: bend each bone and mark in red the faces that stretch or tear
`q` `e`	turn the view by steps
`z`	frame the model

In skin mode: arrows choose a face, space marks it, `c` marks the connected ones, `a` gives the marked faces to the bone, PgUp and PgDn change the bone, Backspace or Del clears the marks, `v` goes back to the bones.

F3, animate

Key	What it does
`n`	new animation
PgUp, PgDn	previous, next animation
up, down	choose the bone
left, right	one frame back, forward
Home, End	start, end of the animation
space	play, stop
`w` `s`	turn the bone around x by 15 degrees (uppercase: 5)
`a` `d`	turn the bone around y
`q` `e`	turn the bone around z
`g`	switch between turning and moving the bone
`k`	put a keyframe here
`x` or Del	remove the keyframe here
`,` `.`	go to the previous, next keyframe
`(` `)`	move the keyframe by one frame
`Ctrl+C`, `Ctrl+V`	copy, paste the pose
`r`	put the bone back at rest
`m`	mirror the pose
`o`	onion skin: the bones of the previous and next keyframes
`l`	loop on, off
`i`	linear, smooth, step
`<` `>`	shorter, longer by one frame
Enter	rename the animation (up to 16 letters)
`Ctrl+D`	duplicate the animation
Backspace twice	delete the animation

F4, sprites

Key	What it does
up, down	choose a setting
left, right	change it
Enter	draw the sprites and put them in the sheet

Gamepad

Buttons	What they do
Y + left, right	change page
Y + B	the menu
Y + up, down	change model
Y + A	undo
F1: X + d-pad	turn the camera
F2: left, right	turn the camera (X + left, right does nothing)
F3, F4	no camera on the pad
F1: A	play, stop
F1: B	show the skeleton
F2: A + d-pad	move the chosen end (left, right, up, down)
F2: X + up, down	move the chosen end nearer, farther
F2: B	head or tail
F3: A + d-pad	turn the bone around y (left, right) and x (up, down)
F3: X + left, right	turn the bone around z
F3: X + up, down	previous, next animation
F3: B	play, stop
X alone	F1: camera spin; F2: new bone; F3: keyframe (or new animation if there is none)
F4: A	draw the sprites

Caveats

- **[TO FILL: the mouse. The suite's guide says Animator does not use it yet, but the code has mouse handlers (click a bone, drag to move or turn, click the timeline); check on the console]**

Bones and rigging

What it does

A skeleton is a list of bones. Each bone has a head, a tail and a parent: when the parent turns, the child follows. Faces are tied to bones (the skin), so when a bone moves, its faces move with it.

Why it matters

The skeleton is what lets one model have many animations at small cost: the game only moves the bones.

Good to know

- The model must have faces: build it in bm Studio first.
- **n** on an empty model makes one bone, `root`, from the bottom of the model up to half its height. Every next **n** adds a child of the chosen bone, half a unit long in the same direction.
- At most **64** bones in a skeleton.
- Joints at the same point move together: a chain stays joined when you move a joint.
- A shoulder or a hip is not at the end of its parent. Make the bone with **n**, move its tail, then choose its head with **Tab** and press **j**. Now **w a s d r f** move only this head. The parent stays the same, the tail stays, and the animations keep their keys. A bone cannot be shorter than 1/16. Press **j** again to join the joints. Each move is one step of **Ctrl+Z**.
- **x** deletes a bone: its children go to its parent and its faces follow the parent (or the first bone). Deleting the only bone removes the skeleton.
- **m** copies the bone and its children to the other side (x mirrored). The names end in `.L` and `.R`; a bone with no side yet gets `.L` if it is on the positive x side. The poses of the animations are mirrored too. The faces on the other side do not move by themselves: run the skin again with **k** or give them with **a**.
- The parent of a bone is chosen among the bones before it in the list.

The skin

- **k** gives each face to its nearest bone: rigid parts, good for robots and boxy characters.
- **K** gives each corner to its nearest bone: the model stretches at the joints.
- In skin mode (**v**), if you have marked faces, **k** and **K** work only on them.
- **t** tests the skin. It bends each bone in turn by 45 degrees (around x and around z) and marks in red the faces that break: a face whose edge grows to more than 1.6 times its length (it stretches), or a corner shared with a face of another bone that comes apart by more than 2% of the model's height (the skin tears). The status line says how many faces are red and which bone's bend marks most, or that the skin holds. Press **t** again to switch the test off. Then choose the red faces with **v** and give them to a bone with **a**.
- Auto skin (**k**, **K**) switches the test off.

[TO FILL: shots/editor-animator-rig.png]

Try it, step by step

1. Open a model with no skeleton. Go to **F2**. The panel says `NO SKELETON`.
2. Press **n**. The status line says `a skeleton: one bone, root`.
3. Press **n** again. A new bone appears at the tail of `root` and the tail is the end you move.

4. Press **d** and **w** a few times to place its tail. Each press moves it by 1/8.
5. Press **Enter**, type a name such as `arm.L` and press Enter.
6. Press **k**. The status line says `every face follows its nearest bone`.

TRY IT

Choose `arm.L` and press **m**. The status line says `2 bones mirrored` if the arm has one child, and a bone `arm.R` appears on the other side. Press **k** again to give the faces to the new bones.

Keyframes and clips

What it does

An animation (a clip) is a set of keyframes on a timeline: a keyframe stores the pose of all the bones at one moment, and bm Animator fills in what is between two keyframes.

Why it matters

A walk cycle is a handful of poses. You make the poses, bm Animator makes the movement.

Good to know

- The timeline runs at **12 frames per second**.
- A new animation (**n**) is called `anim1`, `anim2` ... It lasts 1 second (12 frames), loops, and has a keyframe at the start with the rest pose.
- **Every turn of a bone is a keyframe** at the current time: turn the bone and the keyframe is there.
- **g** switches the same keys from turning to moving the bone (a small step; nothing else changes).
- **Loop** on: after the last keyframe the animation goes on to the first one, over the rest of the length. A looping animation needs no keyframe at its last frame.
- **i** chooses how the movement goes between two keyframes: linear, smooth (slow at both ends) or step (the pose jumps).
- **<** and **>** shorten and lengthen the animation by one frame. It cannot become shorter than its last keyframe.
- `,` and `.` jump between keyframes. `(` and `)` move the keyframe you are on by one frame.
- The only keyframe left of an animation cannot be removed (`the last keyframe stays`).
- Pressing **Backspace** once asks to confirm; a second press deletes the animation.
- At most **255** animations for a model.

Onion skin

o shows the bones of the previous keyframe and of the next one beside the current pose, in two colours. Use it to see where the movement comes from and where it goes.

[TO FILL: shots/editor-animator-animate.png]

Try it, step by step

1. Go to **F3** with a model that has a skeleton. If the panel says `NO SKELETON`, make one in F2 first.
2. Press **n**. The status line says `new animation anim1`.
3. Choose a bone with up and down.
4. Press **w** twice. The bone turns and a diamond appears on the timeline: that is a keyframe at frame 0.
5. Press **right** six times to go to frame 6, then **s** four times to turn the bone the other way. A second diamond appears.
6. Press **space**. The animation plays and loops.
7. Press **Ctrl+S** to save.

TRY IT

Press **o** and step with `,` and `.` : the coloured bones show the poses before and after.

Hitboxes and baking sprites (F4)

What it does

For 2D games, the **sprites** page draws an animation with the console's 3D engine and puts the frames into the **sprite sheet**, ready for `zspr()`. It can also make the **boxes** of every frame from the bones.

[TO FILL: shots/editor-animator-sprites.png]

The settings

Up and down choose a row, left and right change it.

Row	Values
animation	one of the model's animations (or the rest pose if it has none)
frames	1 to 16
size	16, 24, 32, 48, 64, 96 or 128 pixels
directions	1, 2, 4 or 8
from above	0 to 75 degrees, in steps of 15
camera	flat or perspective
light	on, or off (flat colours)
outline	dark or none
colours	all, 32, 16 or 8
fill	50 % to 160 %: how much of the frame the model fills
hitboxes	from bones, or none

The preview on the right turns through the directions while the animation plays, at its size, and also twice as big when the size is 64 or less. Under the settings the screen says how many pixels the sprites will take in the sheet.

The bone hitboxes

With **hitboxes** on, every bone becomes a box in every frame: a rectangle around the head and the tail of the bone as drawn in that frame, with a small margin. The kind of the box comes from the **name of the bone**:

The name contains	Kind	Outline in the preview
`hit` , `weapon` , `punch` , `fist` , `sword` , `blade` , `spear` , `attack`	hit: where it hurts	red
`root` , `body` , `hips` , `pelvis`	body: the room the body takes	green
anything else	hurt: where it can be hit	blue

Name your bones with this in mind: a bone called `sword` makes a hitbox, a bone called `arm.L` makes a hurtbox.

Putting the sprites in the sheet

Press **Enter**. The screen says `drawing the sprites...` while it works, in steps: drawing, outline, colours, then putting them in the sheet.

- The sprites go **below what the sheet already has**, enlarging the sheet if needed (never past 4096 pixels).
- One row of frames per direction: frame `f` of direction `d` is at `x0 + f * size`, `y0 + d * size`.
- The page adds a **zone** to the sheet, named after the model and the animation in small letters (`model_animation`, up to 16 characters), with the frame count and the speed (frames divided by the animation's length). A zone with the same name is replaced. If it was a stack zone, it keeps its slices when the new zone still fits the sheet.
- The boxes are made in the editor session (the cartridge's BOXES section), for the zone, frame by frame (from frame 1), taken from the first direction. It was not confirmed that Ctrl+S writes them to the file.
- A line at the bottom shows the zone and the `sspr()` call to draw the frames.

Good to know

- The total size must stay under 512 x 512 pixels (frames times size by directions times size). If it does not, the screen says `too many, fewer frames or a smaller size`.
- With a looping animation the frames are spread over the length without repeating the first at the end; without loop, the last frame is the last pose.
- Ctrl+Z undoes sprites placed into the sheet.

Try it, step by step

1. Go to **F4** with a model that has an animation.
2. Choose the animation in the first row, `frames` 8, `size` 48, `directions` 4.
3. Check that `hitboxes` says `from bones`. The preview shows the boxes with their coloured outlines.
4. Press **Enter**.
5. Wait for the message `N sprites[ and M boxes] put in sheet at x,y[ (the sheet is bigger now)]`; the position ends it only if the sheet did not grow. The line at the bottom shows the zone name.
6. Press **Ctrl+S** to save.

TRY IT

Rename a bone to `sword` in F2, bake again and look at the preview: its box turns red.

Saving and exporting

What it does

Ctrl+S writes your work into the project file. Only the MESH and ANIM sections change (and the sprite sheet, if you put sprites in it). It was not confirmed whether the SPRITES and BOXES sections (the zones and boxes made in the session) are written too. The game's code, map, cover and sounds stay byte for byte.

Good to know

- A file opened and saved with no changes stays identical.
- **Save as** (`Ctrl+Shift+S`) asks for an 8.3 name; the file goes to `/bme` with the `.BME` ending.
- A game (`.bm` , `.b16`) is never overwritten: the first save makes a `.BME` copy.
- **F5** saves first and plays the project. Start + Select (or Ctrl+Esc) leaves the game and brings you back to the same page.
- To make the finished game, use **bm SDK** (Build, `Ctrl+B`): it writes the `.bm` . The menu of bm Animator has no build entry.
- When the animation is for a game kept in the repository, the models and skeletons are saved by the console and put in `carts/<game>/` as `models.bm` ; `make` takes them into the cartridge.

Try it, step by step

1. Press **Esc**. The menu lists Continue, Open, Save, Save as, Try the game, Open in bm Studio and Exit.
2. Choose **Save as...** and type a name such as `HERO` .
3. The status line says `saved` and the path. The title of the menu shows the file without `*modified*` .

Import a .glb

The SD card has two folders for models. `/glb` holds `.glb` files that you copy there; bm Animator only reads them. `/bmm` holds the converted models (`.bmm`); bm Animator writes them there. All the names are in lower case.

1. Press **Ctrl+I**, or choose **Import a model...** in the menu (Esc). On the pad, press Y + B for the menu.
2. The list shows `/bmm` and `/glb` . A name marked `.glb: converted to /bmm` has no `.bmm` yet.
3. Choose a `.bmm` to read it. Choose a `.glb` to convert it: bm Animator writes `/bmm/NAME.bmm` and adds the model to the project.
4. If the name has more than 8 characters, or `/bmm` has the name already, bm Animator asks for a new name (1 to 8 of a-z, 0-9, `_` and `-`).
5. The status line shows the vertices and the triangles. If the file is not good, it shows the message of the console, for example the format limit of 65535 vertices or triangles.

A `.glb` gives flat colours and no skeleton: make one on the rig page (F2). A `.bmm` with a skeleton keeps it, and its bones show in the status line. The texture of a `.bmm` goes on the sheet of the project only when the sheet is empty; if not, the faces that used it turn grey. A `.bmm` with several models gives the first one.

Ctrl+E, or **Export to /bmm...** in the menu, writes the current model to `/bmm/NAME.bmm` with its skeleton and animations; if the file is there, bm Animator asks before it replaces it.

Using the animation in a game

All these calls are in the 3D API (`docs/api/light3d.md`). The animation is part of the cartridge: no extra file.

The 3D model

Call	What it does
`models()`	the names of the models in the cartridge
`model(name)`	the model as a mesh; do it in `_init`
`clips(m)`	the animations of a model: `{ {name=, length=, loop=}, ... }`
`animate(m, [anim, t, anim2, t2, k, bone])`	puts the model in the pose of animation `anim` (name or number) at time `t` in seconds; with `anim2, t2, k` it blends two animations; returns the length
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	draws the posed model
`bones3d(m)`	the names of the bones
`bone3d(m, bone)`	where the head and the tail of a bone are in the last pose: hands, a sword's tip, a light
`bone_turn(m, bone, [rx, ry, rz])`	turns a bone on top of the animation: aiming
`hit3d(m, x, y, z, ry, scale, ox, oy, oz, dx, dy, dz, [maxd])`	a ray against the bones in the last pose; gives the distance and the bone

`animate` costs only the bones. Call it before `draw3d` , every frame.

The 2D sprites

Call	What it does
`zspr(name, x, y, [frame, flip_x, flip_y, zoom])`	draws the zone made by F4; without `frame` it runs by itself at the zone's speed
`zone(name)`	`x, y, w, h, frames, fps, slices` of the zone; `slices` is 0 unless it is a stack zone
`zboxes(name, [frame, kind])`	the boxes of the zone: `{x, y, w, h, kind, frame}` , with `kind` `"hurt"` , `"hit"` or `"body"`

`zspr` draws the first direction. For the other directions use `sspr()` with the numbers on the line at the bottom of F4.

Code to copy

COPY AND PASTE

```
local v, t = nil, 0
function _init()
  v = model("villager")
end

function _update()
  t = t + 1 / 60
end

function _draw()
  cls(1)
  zclear()
  camera3d(0, 1, -4)
  animate(v, "walk", t)
  draw3d(v, 0, 0, 0, 0, 0, 0, 1)
end
```

This is the pattern of the repository's `carts/village`. For the 2D sprites:

COPY AND PASTE

```
zspr("hero_walk", 100, 80)
for _, b in ipairs(zboxes("hero_walk", 3, "hit")) do
  rect(100 + b.x, 80 + b.y, b.w, b.h, 8)
end
```

Tutorials

Rig a model

1. Open a `.bme` with a biped model (made in bm Studio) and go to **F2**.
2. Press **n**: `root` appears, from the bottom of the model up to half its height.
3. Move the tail of `root` to the hips with **w** or **s** (each press is 1/8).
4. Press **n** to add a bone, name it `leg.L` with Enter, and place its tail on the left foot with **s**, **a** and **d**.
5. Press **m** to make `leg.R` on the other side.
6. Add arms and a head the same way: `arm.L`, `arm.R`, `head`.
7. Press **k**: every face follows its nearest bone. Press **K** instead if you want soft joints.
8. Go to **F1** and press **k**: the skeleton is drawn over the model.

BEWARE
If a part of the model follows the wrong bone, go back to F2, press **v**, choose its faces with space, choose the right bone with PgUp and PgDn, and press **a**.

Make a walk cycle

1. In **F3** press **n**: `anim1` with 12 frames and loop on. Press Enter and rename it `walk`.
2. Choose `leg.L` with up and down. Press **w** twice: the leg turns around x by 30 degrees.
3. Choose `leg.R`. Press **s** twice: the leg turns the other way. Do the same with `arm.L` and `arm.R`, the arms against the legs. This is the first pose, at frame 0.
4. Press **right** six times to go to frame 6.
5. Press **m**: the pose is mirrored, with the left and right sides exchanged. This is the second pose.
6. Press **space**: the animation loops, going from the second pose back to the first over the other six frames.
7. Press **i** until the top line says `smooth`. Press **o** to check the poses with the onion skin.
8. Press **Ctrl+S**.

TRY IT
Lengthen the animation with `>` to slow the walk, shorten it with `<` to speed it up.

Play it in a game

1. In bm SDK, open the project and go to the 3D page (F4): the model turns with its animations.
2. Press **i** there: the SDK writes in the code the lines to load the model and to draw it with `animate` and `draw3d`.
3. Press **F5** to try the game. The model walks.
4. Change the animation by name: `animate(v, "walk", t)`, with `t` the time in seconds.

5. To go from one animation to another smoothly, give two animations and a blend: `animate(v, "walk", t, "wave", t, k)`, with `k` going from 0 to 1.
6. Press Start + Select to leave the game and go back to the editor.

BEWARE

`animate` gives an error if the model has no skeleton or if the animation name is not there. Check with `clips(v)`.

Tips and limits

Tips

- Name the bones for the job: `.L` and `.R` for the two sides, `sword` or `fist` for the hitboxes of an attack, `root` , `body` , `hips` for the body.
- Make the pose at the two extremes of a movement and let the loop fill between: a walk needs only two poses.
- Use onion skin when a movement looks wrong.
- Turn the light off in F4 (flat colours) and use 16 or 8 colours for a hand-drawn look.
- Keep the model small: the console draws about 1200 triangles at 60 frames per second.

Limits

What	Limit
bones in a skeleton	64
animations in a model	255
length of a bone or an animation name	16 letters
timeline	12 frames per second
frames in a sprite animation	16
sprite size	16 to 128 pixels
sprites in one bake	512 x 512 pixels in all
the sprite sheet	4096 x 4096 pixels
skinning	each corner follows one bone (no mixed weights)

What bm Animator does not do

- There are no event markers on the timeline (a hit frame, a sound): the game must count the frames itself.
- There are no socket points: the 2D position of a hand or of a muzzle is not exported with the sprites. In 3D use `bone3d()` .
- The sprites are drawn as one block: there are no separate layers for armour or weapons.
- The sprite names have no direction suffixes: one zone per animation, directions in rows.

Caveats

- **[TO FILL: how many undo steps are kept]**
- **[TO FILL: how long a bake takes on the console with 8 directions at 128 pixels; not timed yet]**

Open points

- **The screen and the controls**: picture to take: shots/editor-animator-play.png
- **The screen and the controls**: the mouse. The suite's guide says Animator does not use it yet, but the code has mouse handlers (click a bone, drag to move or turn, click the timeline); check on the console
- **Bones and rigging**: picture to take: shots/editor-animator-rig.png
- **Keyframes and clips**: picture to take: shots/editor-animator-animate.png
- **Hitboxes and baking sprites (F4)**: picture to take: shots/editor-animator-sprites.png
- **Tips and limits**: how many undo steps are kept
- **Tips and limits**: how long a bake takes on the console with 8 directions at 128 pixels; not timed yet